Bryce Tabangcura - Lab Week whatrever

Lab partner: haven bunter-gooley

Pre-Lab

1. gamma ray radiation Well theoretically air has its own gamma absorption, But the biggest contributor to a lower intensity is likely the same way light intensity lowers as a matter of disntace, because its spread farther.

So going by that

$$I_{gamma} \propto \frac{1}{x^2}$$

2. Mass attenuation solve simple equation thingy

$$\mu = \rho \times \sigma$$

For 75% to come out you'd expect only 25% to travel out. so you assume $I(x) = 0.75I_0$

$$0.75I_0 = I_0 e^{-\mu x}$$

$$0.75 = e^{-\mu x}$$

$$0.75 = e^{-0.1 \times 7.874 \times x}$$

I'm going to skip all of the algebra and go straight ot the answer

$$x = 1.7606\ cm$$

Preamble

Entire experiment done and interpolated in python. Here is the generic swiss army knife helper functions and imports I use for my lab reports. these can be largely ignored. I'm noticing they're only getting bigger and bigger but just ignore the following.

```
import warnings
from math import e, log, pi, sqrt

import matplotlib.pyplot as plt
import numpy as np
from scipy import constants, odr
from scipy.stats import norm
from uncertainties import UFloat, ufloat, umath

warnings.filterwarnings("ignore", category=FutureWarning,
module="uncertainties")
```

```
warnings.filterwarnings(
    "ignore",
    category=FutureWarning,
    message=r"AffineScalarFunc\.__abs__\(\) is deprecated",
)
warnings.filterwarnings(
    "ignore", category=UserWarning, message="Using UFloat objects with
std_dev==0"
)


def fit_odr(x: list[UFloat], y: list[UFloat], zero_intercept: bool = False):
    x_nom, x_err = [v.n for v in x], [v.s for v in x]
    y_nom, y_err = [v.n for v in y], [v.s for v in y]
    data = odr.RealData(x_nom, y_nom, sx=x_err, sy=y_err)
    if zero_intercept:
        model, beta0 = odr.Model(lambda p, x: p[0] * x), [1]
    else:
        model, beta0 = odr.Model(lambda p, x: p[0] * x + p[1]), [1, 0]
    out = odr.ODR(data, model, beta0=beta0).run()
    slope = ufloat(out.beta[0], out.sd_beta[0])
    intercept = (
        ufloat(out.beta[1], out.sd_beta[1]) if not zero_intercept else
ufloat(0, 0)
    )
    y_pred = model.fcn(out.beta, np.array(x_nom))
    ss_res = np.sum((np.array(y_nom) - y_pred) ** 2)
    ss_tot = np.sum(
        np.array(y_nom) ** 2
        if zero_intercept
        else (np.array(y_nom) - np.mean(y_nom)) ** 2
    )
    r_squared = 1 - ss_res / ss_tot
    return slope, intercept, r_squared


def compsigma(a, b):
    return abs((a.n - b.n) / ((((a.s**2) + (b.s**2)) ** (1 / 2))))


def plot(
    x,
    y,
    fitodr=None,
    title="default title",
    xlabel="default xlabel",
    ylabel="default ylabel",
    show=True,
```

```
):
    if fitodr is None:
        fitodr = fit_odr(x, y)
    slope, intercept, r_squared = fitodr
    plt.figure()
    first = x[0].n
    last = x[-1].n

    plt.plot(
        [first, last],
        [first * slope.n + intercept.n, last * slope.n + intercept.n],
        "r-",
        label=f"Fit: $y = ({slope.n:.4g})x + ({intercept.n:.4g}),R^2 =
{r_squared:.4f}$",
    )

    plt.errorbar(
        [v.nominal_value for v in x],
        [v.nominal_value for v in y],
        xerr=[v.std_dev for v in x],
        yerr=[v.std_dev for v in y],
        capsize=5,
        linestyle="none",
        marker="o",
        markersize=3,
        label="Data",
    )

    plt.title(title)
    plt.xlabel(xlabel)
    plt.ylabel(ylabel)
    plt.legend()
    if show:
        plt.show()


def plot_residuals(
    x,
    y,
    fitodr=None,
    title="Residuals",
    xlabel="default xlabel",
    ylabel="Residual (y - fit)",
    show=True,
):
    if fitodr is None:
        fitodr = fit_odr(x, y)
    slope, intercept, r_squared = fitodr
```

```
    residuals = [yi - (slope * xi + intercept) for xi, yi in zip(x, y)]
    sigma_eff = [
        sqrt(yi.std_dev**2 + (slope.n * xi.std_dev) ** 2) for xi, yi in zip(x,
y)
    ]
    chi2 = sum((r.nominal_value**2) / (s**2) for r, s in zip(residuals,
sigma_eff))
    dof = len(y) - 1
    chi2_reduced = chi2 / dof
    zero_fit = (
        ufloat(0, 0),
        ufloat(0, 0),
        0.0,
    )  # was None — plot()'s label needs a float

    plot(
        x,
        residuals,
        fitodr=zero_fit,
        title=f"{title} ($\\chi^2_\\nu = {chi2_reduced:.3f}$)",
        xlabel=xlabel,
        ylabel=ylabel,
        show=show,
    )

    return chi2_reduced


def two_tailed_p(diff: float):
    z = abs(diff)
    return 2 * norm.sf(z)
```

Experiment 1

Aim

The aim of this experiment is to experimentally deduce the linear absorption coefficient μ of a miscellaneous barrier.

Method

First is measuring. We used pvc plastic as our barrier, for all three barriers we measured precisely using a calliper. Me and my lab partner both did independent measurements. This is for our own error propagation.

Then we set up our equipment, with a stand, we clamped the geiger counter connected to the lab computer above the lead uncasing. We do not move or touch the position of this geiger counter for all until the next experiment.

We then used 2 minute time intervals to measure the background radiation. we use 2 minute intervals for all measurements. We use seconds as our increments.

After measuring background radiation we chose barium as our radioactive substance, due to its high half life and its reasonably high activity. We inserted it into the lead casing. We recorded the intensity using the same 2 minute interval, and repeated the process with 1 and 2 layers of the same thickness pvc barrier.

The software we were using, vernier had a statistics tab, which had both a mean and a standard deviation, we recorded those only.

Since our results were bad, we attempted this again, but instead we started with three barriers of pvc plastic, and situated the geiger as close as possible to the plastic, putting our geiger close to the substance increases our radiation, which yields better results. We repeated the entire experiment with this closer situated geiger

Also for our first experiment we only had 2 pvc barriers, whereas the second embettered experiment had 3 pvc barriers.

Results

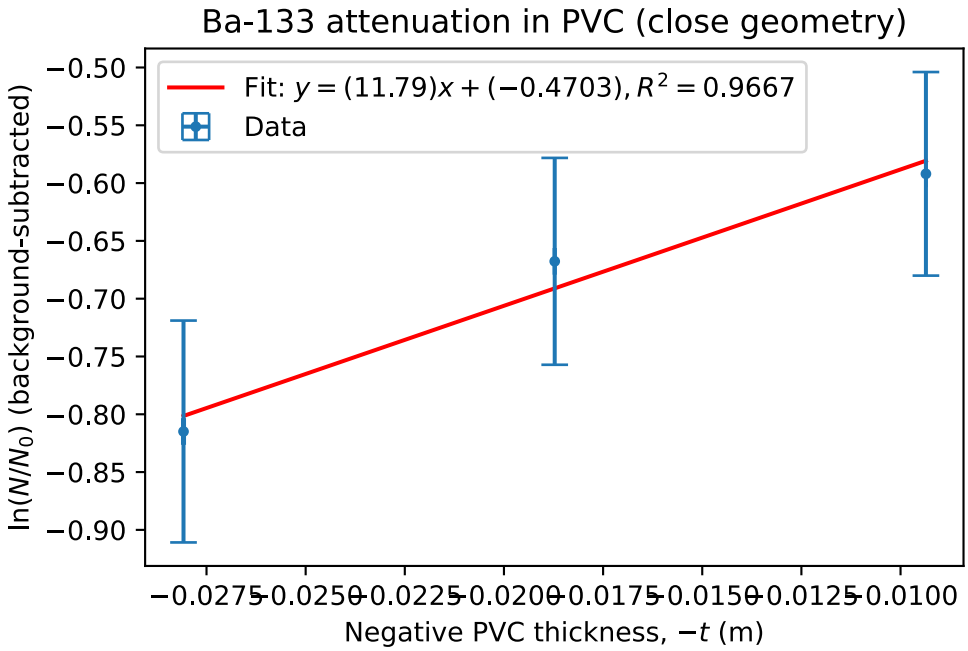
Here is our data below. List of tuples, first datapoint each tuple is the amount of barriers, -1 means background. The second and third values in each tuple are the mean and standard deviation respectively.

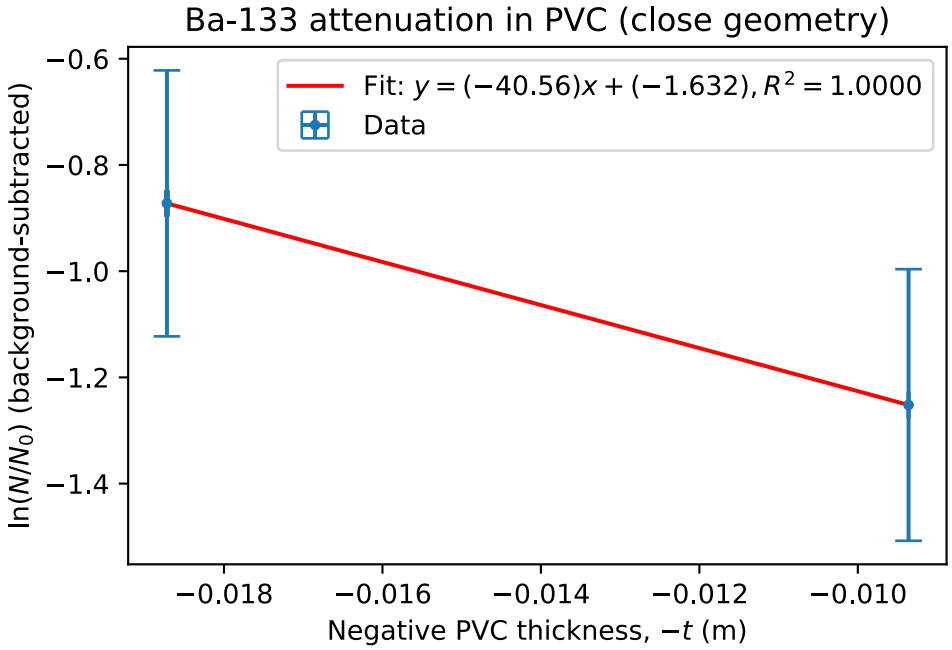
```
exp2data: list[tuple[int, float, float]] = [
    (-1, 0.325, 0.663),
    (-1, 0.408, 0.628),
    (0, 2.225, 1.399),
    (0, 2.425, 1.459),
    (1, 1.433, 1.207),
    (1, 1.467, 1.223),
    (2, 1.242, 1.045),
    (2, 1.5, 1.23),
    (3, 1.317, 1.123),
    (3, 1.150, 1.001),
]
exp1data: list[tuple[int, float, float]] = [
    (-1, 0.574, 0.3),
    (-1, 0.549, 0.274),
    (-1, 0.607, 0.376),
    (0, 1.300, 1.213),
    (1, 0.892, 0.968),
    (1, 0.800, 0.885),
    (2, 0.875, 0.975),
    (2, 0.883, 0.852),
    (1, 0.725, 0.860),
    (1, 0.717, 0.842),
```

```
]
lengthpvc = ufloat(9.36, 0.01) / 1000
```

We can also plot this (Ignore the code I pasted code from after to make this work since graphs should be in results)

```
def plotbetter(data: list[tuple]):
    ufloats = [(x[0], ufloat(x[1], x[2] / sqrt(120))) for x in data]
    numbers = set([x[0] for x in ufloats])
    combinedufloats = [list(filter(lambda y: y[0] == x, ufloats)) for x in
numbers]
    values = [(x[0][0], sum([y[1] for y in x]) / len(x)) for x in
combinedufloats]
    sortedvalues = sorted(values, key=lambda x: x[0])
    data = [(x[0], x[1] - sortedvalues[0][1]) for x in sortedvalues[1:]]
    xdata = [-x[0] * lengthpvc for x in data[1:]]
    ydata = [umath.log(x[1] / data[0][1]) for x in data[1:]]
    plot(
        xdata,
        ydata,
        title="Ba-133 attenuation in PVC (close geometry)",
        xlabel="Negative PVC thickness, $-t$ (m)",
        ylabel="$\\ln(N/N_0)$ (background-subtracted)",
    )
    pass
plotbetter(exp2data)
plotbetter(exp1data)
```





Conclusions

I have created two functions for the purpose of data interpolation. They are very functional coated in the way that they operate and I will explain them wordily.

Firstly the tuples such that its of size two and the second value in each tuple is just the ufloat, since ufloat does the error propagation for me.

Then I combine the data based on if its one layer and sum them. So for example I sum all of the 1 pvc layer datapoints. This is automatically error propagated because of ufloats.

I sort this and then remove the background from each value.

```
# Functions not created with ai btw
def databetter(data: list[tuple[int, float, float]]):
    ufloats = [(x[0], ufloat(x[1], x[2] / sqrt(120))) for x in data]
    numbers = set([x[0] for x in ufloats])
    combinedufloats = [list(filter(lambda y: y[0] == x, ufloats)) for x in
numbers]
    values = [(x[0][0], sum([y[1] for y in x]) / len(x)) for x in
combinedufloats]
    sortedvalues = sorted(values, key=lambda x: x[0])
    out = [(x[0], x[1] - sortedvalues[0][1]) for x in sortedvalues[1:]]
    return out
```

Next one is just some math

$$I_0 e^{-\mu x} = I_n$$

$$-\mu x = \ln \frac{I_n}{I_0}$$

and we use $y = \ln \frac{I_n}{I_0}$ and $x = -x$ This is what our next function does, it fits a linear regression line of it and returns the line pretty much.

```
def datainterpolate(data: list[tuple]):
    xdata = [-x[0] * lengthpvc for x in data[1:]]
    ydata = [umath.log(x[1] / data[0][1]) for x in data[1:]]
    out = fit_odr(xdata, ydata)
    return out
```

We can now use these functions to interpolate the data

Barium is in the area of 400KeV gamma radiation. And this corresponds to, assuming pvc plastic is $1.4\frac{g}{cm^3}$. is 14.

I know i'm not sourcing but one source listed pvc plastic as 1.35 ~ 1.45 g/cm^3 so we are using 0.05 as some uncertainty here.

This just corresponds to a μ of $14 \pm 0.5 m^{-1}$

So we compare the sigma value of this to our value.

```
mu2 = datainterpolate(databetter(exp2data))[0]
print(compsigma(mu2, ufloat(14, 0.5)))
```

```
0.959132158446137
```

And we get < 3 sigma, So success on our second experiment, our second experiment is consistent with the real expected value.

unfortunately, for our first experiment, where we had the geiger situated farther away, with only 2 pvc layers, we get a value of:

```
mu1 = datainterpolate(databetter(exp1data))[0]
print(compsigma(mu1, ufloat(1.4, 0.05)*10))
```

```
109.11471057061416
```

This is significantly larger than 3, and indicates a large systematic error in our experiment. The reasons why are likely due to how much background radiation actually matters when you increase the distance of the geiger to the radioactive matter. Luck becomes a huge factor and the noise of the background radiation starts outweighing the radioactive matter by large magnitudes.

Overall Conclusion

One of our experiments showed consistency with the actual value of pvc plastic linear absorption coefficient. and our other less improved experiment had severse systematic error.